SQLD 제61회 기출 복원문제

각 문항 아래에 정답과 선택지별 풀이를 바로 확인할 수 있습니다.

정답·상세 풀이
총 50문항 · 1과목 10 / 2과목 40

1과목 데이터 모델링의 이해

01 1과목

다음 중 외부 스키마에 대한 설명으로 옳지 않은 것은?

① 사용자나 응용프로그램 개발자의 관점에서 필요한 데이터베이스의 논리적 구조이다.

② 하나의 데이터베이스에 대해 여러 개의 외부 스키마가 존재할 수 있다.

③ 데이터베이스 전체의 저장 구조를 정의하며 물리적 저장 방법을 포함한다.

④ 응용프로그램이 필요로 하는 데이터만 정의하여 접근 범위를 제한할 수 있다.

정답 ③

핵심 해설

정답은 ③ 데이터베이스 전체의 저장 구조와 물리적 저장 방법을 정의한다는 설명이다. 이것은 외부 스키마가 아니라 내부 스키마에 대한 설명이다. ANSI/SPARC 3단계 구조에서 외부 스키마는 사용자 관점, 개념 스키마는 통합된 전체 관점, 내부 스키마는 물리적 저장 관점의 구조를 정의한다. 따라서 저장 구조·물리적 저장 방법을 포함한다는 내용은 외부 스키마와 맞지 않는다. 외부 스키마는 서브스키마 또는 사용자 뷰라고도 부르며, 저장 장치나 레코드 배치 같은 물리적 사항은 다루지 않는다.

선택지별 풀이

① 외부 스키마는 사용자나 개발자가 바라보는 데이터베이스의 논리적 구조(사용자 뷰)이므로 옳은 설명이다.

② 사용자·응용프로그램마다 필요한 관점이 다르므로 하나의 데이터베이스에 여러 개의 외부 스키마가 존재할 수 있다. 옳은 설명이다.

④ 외부 스키마는 응용프로그램이 필요로 하는 데이터만 보여 주므로 접근 범위를 제한하는 효과가 있다. 옳은 설명이다.

> **보충 개념** 외부 스키마는 여러 개, 개념 스키마와 내부 스키마는 데이터베이스당 하나다. 외부-개념 사이의 독립성은 논리적 독립성, 개념-내부 사이의 독립성은 물리적 독립성이다.

02 1과목

다음 설명이 나타내는 것으로 가장 적절한 것은?

설명

각 속성이 가질 수 있는 값의 범위로, 데이터 타입·길이·허용값·제약 사항 등을 함께 정의한다.

① 도메인 ② 식별자 ③ 엔터티 ④ 인스턴스

정답 ①

핵심 해설

정답은 ① 도메인이다. 도메인은 각 속성이 가질 수 있는 값의 범위로, 데이터 타입·길이·허용값·제약 사항 등을 함께 정의한다. 예를 들어 성별 속성의 도메인은 '남, 여', 나이 속성의 도메인은 '0 이상의 정수'처럼 정할 수 있다. 제시문의 "속성이 가질 수 있는 값의 범위"는 도메인의 정의 그대로다. 도메인을 정의해 두면 속성에 잘못된 값이 들어오는 것을 막을 수 있으며, 물리 설계 단계에서는 데이터 타입, 길이, CHECK 제약조건 등으로 구현된다.

선택지별 풀이

② 식별자는 엔터티 내의 각 인스턴스를 유일하게 구분하는 속성(또는 속성의 조합)으로, 값의 범위를 뜻하지 않는다.

③ 엔터티는 업무에 필요하고 관리해야 할 정보의 집합(예: 고객, 주문)으로, 속성을 담는 그릇이지 값의 범위가 아니다.

④ 인스턴스는 엔터티에 속한 개별 데이터 한 건(예: 고객 '홍길동')을 의미한다.

> **보충 개념** 엔터티 > 인스턴스 > 속성 > 속성값의 계층 구조와, 각 속성마다 하나의 도메인이 정해진다는 점을 함께 정리해 두면 혼동이 적다.

03 1과목

엔터티를 유형/무형 엔터티 기준으로 분류할 때, 다음 중 이 분류 기준에 해당하지 않는 것은?

① 유형 엔터티 ② 개념 엔터티 ③ 사건 엔터티 **④ 행위 엔터티**

정답 ④

핵심 해설

정답은 ④ 행위 엔터티이다. 엔터티를 유무형(물리적 형태의 존재 여부)에 따라 분류하면 유형 엔터티, 개념 엔터티, 사건 엔터티의 세 가지로 나뉜다. 행위 엔터티는 유무형 기준이 아니라 발생 시점에 따른 분류(기본/키 엔터티, 중심 엔터티, 행위 엔터티)에 속한다. 즉 유무형 기준과 발생 시점 기준은 서로 다른 분류 축이므로, 문제에서 어떤 기준을 묻는지부터 확인해야 한다.

선택지별 풀이

① 유형 엔터티는 물리적 형태가 있고 안정적이며 지속적으로 활용되는 엔터티(예: 사원, 물품)로, 유무형 기준 분류에 해당한다.
② 개념 엔터티는 물리적 형태는 없지만 관리해야 할 개념적 정보(예: 조직, 보험상품)로, 유무형 기준 분류에 해당한다.
③ 사건 엔터티는 업무 수행 과정에서 발생하며 발생량이 많은 엔터티(예: 주문, 청구)로, 유무형 기준 분류에 해당한다.

> **보충 개념** 발생 시점 기준 분류는 기본 엔터티(독립적으로 생성, 예: 고객) → 중심 엔터티(기본에서 발생, 예: 주문) → 행위 엔터티(두 개 이상 부모에서 발생, 예: 주문내역)의 순서로 기억하면 된다.

04 1과목

다음 중 비식별 관계로의 전환을 고려해야 하는 상황으로 가장 적절한 것은?

① 자식 엔터티가 부모 엔터티에 강하게 의존하며 부모 없이는 존재 의미가 없는 경우
② 상속받은 식별자가 계속 연결되어 주식별자의 구성 속성 수가 많아지고 복잡해지는 경우
③ 자식 엔터티의 등록번호를 반드시 부모 식별자와 연계해서 관리해야 하는 경우
④ 이력 관리를 위해 부모의 식별자를 자식이 반드시 포함해야 하는 경우

정답 ②

핵심 해설

정답은 ② 상속받은 식별자가 계속 연결되어 주식별자 구성 속성이 많아지고 복잡해지는 경우이다. 식별 관계로만 계속 연결하면 부모의 주식별자가 자식, 손자 엔터티로 계속 전이되어 하위로 갈수록 주식별자 속성 수가 늘어나고 조인 조건과 SQL이 복잡해진다. 이런 경우 비식별 관계로 끊어 자식이 독립적인 주식별자를 갖도록 설계하는 것을 고려한다. 비식별 관계로 바꾸면 부모 식별자는 자식의 일반 속성(FK)으로만 남고, 자식은 자체 일련번호 등으로 주식별자를 단순하게 구성할 수 있다.

선택지별 풀이

① 자식이 부모에 강하게 의존하고 부모 없이 존재 의미가 없다면 부모 식별자를 주식별자에 포함하는 식별 관계가 적합하다.
③ 자식의 번호를 반드시 부모 식별자와 연계해 관리해야 한다면 부모 식별자가 자식 주식별자의 일부가 되는 식별 관계가 적합하다.
④ 이력 관리 등으로 부모 식별자를 자식이 반드시 주식별자에 포함해야 하는 경우도 식별 관계를 유지하는 상황이다.

> **보충 개념** 비식별 관계를 고려하는 대표 상황은 약한 연결 관계, 자식 엔터티의 독립적인 PK 필요, 주식별자 속성 수 과다, SQL 복잡성 증가이다. 반대로 강한 연결이 필요하면 식별 관계를 쓴다.

05 1과목

다음 중 관계선택사양에 대한 설명으로 옳은 것을 모두 고른 것은?

가. 관계에 참여하는 것이 필수인지 선택인지를 나타낸다.
나. 필수/선택 여부는 조직의 업무 규칙에 따라 결정된다.
다. 두 엔터티 간 관계에서 참여자의 수(1:1, 1:M, M:N)를 표현한다.

① 가 **② 가, 나** ③ 나, 다 ④ 가, 나, 다

정답 ②

핵심 해설

정답은 ② 가, 나이다. 관계선택사양(Optionality)은 관계에 반드시 참여해야 하는지(필수) 참여하지 않아도 되는지(선택)를 나타내는 것이다. IE 표기법의 ERD에서는 필수 참여를 관계선 끝의 세로선(|), 선택 참여를 원(O)으로 표시한다. 가) 관계 참여가 필수인지 선택인지를 나타낸다는 것은 관계선택사양의 정의 그대로이므로 옳다. 나) 필수/선택 여부는 "주문은 반드시 고객이 있어야 한다"와 같은 조직의 업무 규칙에 따라 결정되므로 옳다. 다) 두 엔터티 간 관계에서 참여자의 수(1:1, 1:M, M:N)를 표현하는 것은 관계차수(Cardinality)에 대한 설명이므로 틀리다. 따라서 옳은 항목은 가, 나이다.

선택지별 풀이

① 나)도 옳은 설명이므로 가만 고른 것은 답이 될 수 없다.

③ 다)는 관계차수에 대한 설명이므로 포함될 수 없고, 가)가 빠져 있다.

④ 다)가 포함되어 있으므로 옳지 않다.

> **보충 개념** 관계의 표기법은 관계명, 관계차수, 관계선택사양의 세 가지 개념으로 구성된다. 참여 '수'는 차수, 참여 '여부'는 선택사양으로 구분한다.

06 1과목

데이터 모델링의 관계에 대한 설명으로 가장 적절하지 않은 것은?

① 관계는 존재에 의한 관계와 행위에 의한 관계로 구분될 수 있으나, ERD에서는 관계를 연결할 때 존재와 행위를 구분하지 않고 단일화된 표기법을 사용한다.

② UML은 클래스 다이어그램의 관계 중 연관 관계와 의존 관계가 있으며, 이는 실선과 점선의 표기법으로 다르게 표현된다.

③ 연관 관계는 항상 이용하는 관계로 존재적 관계에 해당하고, 의존 관계는 상대방 클래스의 행위에 의해 관계가 형성되는 행위적 관계에 해당한다.

④ 연관 관계는 오퍼레이션에서 파라미터 등으로 이용할 수 있고, 의존 관계는 소스 코드에서 멤버 변수로 선언하여 사용할 수 있다.

정답 ④

핵심 해설

정답은 ④ 연관 관계와 의존 관계의 구현 방식을 설명한 문장이다. 이 설명은 두 관계의 구현 방식을 서로 바꿔 놓았다. 연관 관계는 항상 이용하는 관계이므로 소스 코드에서 멤버 변수로 선언하여 사용하고, 의존 관계는 상대 클래스의 행위에 의해 일시적으로 형성되므로 오퍼레이션의 파라미터 등으로 이용한다. 연관 관계는 객체가 생성되어 있는 동안 계속 참조하는 구조이고, 의존 관계는 메서드가 실행되는 동안에만 잠깐 참조하는 구조라는 점에서 구현 방식이 갈린다.

선택지별 풀이

① 관계는 존재에 의한 관계(예: 부서-사원)와 행위에 의한 관계(예: 고객-주문)로 구분되지만, ERD에서는 이를 구분하지 않고 단일 표기법으로 연결한다. 옳은 설명이다.

② UML 클래스 다이어그램에서 연관 관계는 실선, 의존 관계는 점선으로 표현한다. 옳은 설명이다.

③ 연관 관계는 항상 이용하는 존재적 관계, 의존 관계는 상대 클래스의 행위에 의해 형성되는 행위적 관계에 해당한다. 옳은 설명이다.

> **보충 개념** 연관 관계 = 실선 = 멤버 변수 = 존재적 관계, 의존 관계 = 점선 = 파라미터 = 행위적 관계로 짝지어 기억하면 된다.

07 1과목

다음 [고객] 엔터티의 속성에 대한 설명으로 옳지 않은 것은?

고객

속성	특징
고객번호	식별 가능, NOT NULL
주민등록번호	식별 가능, NOT NULL
이메일	식별 가능, NULL 허용
고객명	중복 가능, NULL 허용

① 고객번호는 유일성과 최소성을 만족하므로 주식별자(PK)로 사용할 수 있다.

② 주민등록번호는 유일성과 최소성을 만족하므로 주식별자(PK)로 사용할 수 있다.

③ 이메일은 유일성을 만족하므로 주식별자(PK)로 사용할 수 있다.

④ 고객명은 중복과 NULL이 허용되므로 식별자로 사용할 수 없다.

정답 ③

핵심 해설

정답은 ③ 이메일은 유일성을 만족하므로 주식별자(PK)로 사용할 수 있다는 설명이다. 이메일은 식별 가능하지만 NULL을 허용한다. 주식별자는 유일성·최소성뿐 아니라 존재성(NOT NULL)과 불변성도 만족해야 하는데, 이메일이 없는 고객 인스턴스는 식별할 수 없으므로 주식별자로 사용할 수 없다. 이메일은 대체 식별자(보조 식별자) 정도로 활용할 수 있다. 후보 식별자 중 하나를 주식별자로 선정하고, 나머지는 대체 식별자로 관리하는 것이 일반적이다.

선택지별 풀이

① 고객번호는 식별 가능하고 NOT NULL이며 단일 속성으로 최소성도 만족하므로 주식별자로 사용할 수 있다. 옳은 설명이다.

② 주민등록번호도 식별 가능하고 NOT NULL이며 단일 속성이므로 주식별자 후보가 될 수 있다. 옳은 설명이다.

④ 고객명은 중복과 NULL이 허용되어 유일성과 존재성을 모두 만족하지 못하므로 식별자로 사용할 수 없다. 옳은 설명이다.

> **보충 개념** 주식별자의 특징은 유일성, 최소성, 불변성, 존재성 네 가지다. 유일성만 보고 NULL 허용 여부를 놓치는 것이 대표적인 함정이다.

08 1과목

다음 테이블 구조가 위배하고 있는 정규형은?

주문상품

주문ID (PK)	상품ID (PK)	주문수량	주문금액	할인쿠폰코드	할인쿠폰명
202608001	P12345	2	20000	C10	첫구매할인
202608001	P23456	1	15000	C20	신학기할인
202608002	P97589	3	30000	C30	여름세일

① 제1정규형 위배 ② 제2정규형 위배 **③ 제3정규형 위배** ④ 위배하고 있지 않음

정답 ③

핵심 해설

정답은 ③ 제3정규형 위배이다. 테이블의 PK는 (주문ID, 상품ID)의 복합키이며, 정규형은 1NF부터 차례로 확인한다. 1단계(1NF): 모든 컬럼 값이 하나의 원자값만 가지므로 제1정규형을 만족한다. 2단계(2NF): 주문수량, 주문금액, 할인쿠폰코드는 주문ID와 상품ID 조합에 따라 정해진다(같은 주문 202608001 안에서도 상품별로 C10, C20이 다름). PK 일부에만 종속되는 부분함수 종속이 없으므로 제2정규형을 만족한다. 3단계(3NF): 할인쿠폰명은 PK가 아니라 일반 속성인 할인쿠폰코드에 의해 결정된다(C10=첫구매할인, C20=신학기할인, C30=여름세일). 즉 (주문ID, 상품ID) → 할인쿠폰코드 → 할인쿠폰명의 이행 함수 종속이 존재하므로 제3정규형을 위배한다.

선택지별 풀이

① 모든 값이 원자값이므로 제1정규형은 만족한다.

② PK 일부에만 종속되는 속성이 없으므로 제2정규형 위배가 아니다.

④ 할인쿠폰명의 이행 종속이 있으므로 위배하지 않는다는 설명은 틀리다.

> **보충 개념** 해결 방법은 [할인쿠폰](할인쿠폰코드 PK, 할인쿠폰명) 테이블을 분리하고 주문상품에는 할인쿠폰코드만 남기는 것이다. 일반 속성이 다른 일반 속성을 결정하면 3NF 위배로 판단한다.

09 1과목

다음 중 배타적 관계에 대한 설명으로 가장 적절한 것은?

① 하나의 엔터티가 여러 자식 엔터티와 동시에 관계를 맺을 수 있음을 의미한다.

② **하나의 엔터티가 여러 개의 다른 엔터티 중 하나와만 관계를 맺을 수 있음을 의미한다.**

③ 두 엔터티가 반드시 일대일 관계를 맺어야 함을 의미한다.

④ 자식 엔터티가 부모 엔터티 없이 독립적으로 존재 가능함을 의미한다.

정답 ②

핵심 해설

정답은 ② 하나의 엔터티가 여러 개의 다른 엔터티 중 하나와만 관계를 맺을 수 있다는 설명이다. 배타적 관계(Exclusive Relationship)는 부모 엔터티가 둘 이상일 때, 자식 인스턴스가 그중 하나의 부모와만 관계를 맺는 것을 말한다. 예를 들어 [계좌]는 [개인고객] 또는 [법인고객] 중 하나에만 속하며, 두 부모와 동시에 관계를 맺지 않는다. 즉 하나의 자식 인스턴스 입장에서 여러 부모 엔터티 중 반드시 하나만 선택된다는 것이 핵심이다.

선택지별 풀이

① 여러 엔터티와 '동시에' 관계를 맺는 것은 배타적 관계와 반대되는 포괄적(포함) 관계에 가까운 설명이다.

③ 배타적 관계는 관계차수가 1:1이어야 한다는 제약이 아니다. 차수와 배타성은 별개의 개념이다.

④ 자식이 부모 없이 독립적으로 존재할 수 있다는 것은 관계선택사양(선택 참여) 또는 비식별 관계에 대한 설명이다.

> **보충 개념** ERD에서 배타적 관계는 관계선들을 호(arc)로 묶어 표시하며, 물리 구현 시에는 부모 구분 컬럼과 FK를 하나로 통합하거나 FK를 각각 두는 방식이 쓰인다.

10 1과목

트랜잭션의 특징 중 원자성에 해당하는 것으로 가장 적절한 것은?

① **트랜잭션에 포함된 연산은 논리적으로 하나로 묶여 수행되며, 일부 연산만 반영되는 일이 없어야 한다.**

② 트랜잭션 수행 전과 후에 데이터베이스는 항상 일관된 상태를 유지해야 한다.

③ 동시에 여러 트랜잭션이 수행되어도 서로의 연산 결과에 영향을 주지 않아야 한다.

④ 트랜잭션이 성공적으로 완료되면 그 결과는 영구적으로 반영되어야 한다.

정답 ①

핵심 해설

정답은 ① 트랜잭션의 연산이 논리적으로 하나로 묶여 수행되며 일부만 반영되는 일이 없어야 한다는 설명이다. 원자성(Atomicity)은 트랜잭션의 연산이 모두 성공적으로 실행되거나(Commit) 전혀 실행되지 않은 상태로 남아야 한다(Rollback)는 All or Nothing 성질이다. 계좌이체에서 출금만 되고 입금이 안 되는 상황을 막는 것이 원자성이다. 원자성은 DBMS의 회복(Recovery) 기능과 로그를 통한 ROLLBACK으로 보장된다.

선택지별 풀이

② 수행 전과 후에 데이터베이스가 일관된 상태를 유지해야 한다는 것은 일관성(Consistency)이다.

③ 동시에 수행되는 트랜잭션이 서로의 연산 결과에 영향을 주지 않아야 한다는 것은 고립성(Isolation)이다.

④ 성공적으로 완료된 결과가 영구적으로 반영되어야 한다는 것은 지속성(Durability)이다.

> **보충 개념** ACID는 원자성(A)·일관성(C)·고립성(I)·지속성(D)의 약자다. 고립성이 보장되지 않으면 Dirty Read, Non-Repeatable Read, Phantom Read 같은 현상이 발생한다.

2과목 SQL 기본 및 활용

11 2과목

다음 중 데이터베이스에서 데이터를 저장하는 기본 단위로 가장 적절한 것은?

① **테이블** ② 뷰 ③ 인덱스 ④ 시퀀스

정답 ①

핵심 해설

정답은 ① 테이블이다. 테이블은 행(Row)과 열(Column)의 2차원 구조로 데이터를 실제로 저장하는 관계형 데이터베이스의 가장 기본적인 저장 단위이다. 하나의 테이블은 하나의 엔터티(릴레이션)에 대응하며, 각 행은 하나의 인스턴스, 각 열은 하나의 속성을 표현한다. 다른 객체들은 모두 테이블을 바탕으로 동작하는 보조 객체이다.

선택지별 풀이

② 뷰는 SELECT 문으로 정의된 가상 테이블로, 정의(SQL)만 저장할 뿐 데이터를 자체적으로 저장하지 않으므로 기본 저장 단위가 아니다.

③ 인덱스는 테이블의 특정 컬럼 값과 ROWID를 정렬해 보관하여 검색 속도를 높이는 접근 구조로, 데이터 저장 단위가 아니라 테이블에 종속된 보조 객체이다.

④ 시퀀스는 호출할 때마다 일련번호를 생성해 주는 객체로, 주로 PK 값 채번에 쓰이며 데이터를 저장하는 단위가 아니다.

> **보충 개념** 뷰는 보안 강화(필요한 컬럼만 노출)·SQL 단순화·독립성 제공이 장점이지만 실데이터는 원본 테이블에 있다는 점이 자주 출제된다. 인덱스는 조회는 빨라지지만 INSERT·UPDATE·DELETE 시 인덱스도 함께 갱신되어 DML 부하가 늘어난다.

12 2과목

다음 SQL 실행 결과로 가장 적절한 것은? (결과는 A, B 순)

DEPT

DEPT_ID
1
1
NULL
2
NULL
2
3
3
NULL
3

```
SELECT COUNT(*) AS A, COUNT(DEPT_ID) AS B
FROM DEPT;
```

① 7, 7 ② 10, 10 ③ 7, 10 **④ 10, 7**

정답 ④

핵심 해설

정답은 ④ 10, 7이다. COUNT(*)는 NULL 여부와 상관없이 전체 행 수를 세고, COUNT(컬럼)은 해당 컬럼이 NULL이 아닌 행만 센다. 1단계: DEPT 테이블 전체 행은 10건이므로 A = COUNT(*) = 10이다. 2단계: DEPT_ID가 NULL인 행은 3건(3번째, 5번째, 9번째 행)이므로 B = COUNT(DEPT_ID) = 10 - 3 = 7이다. (값 분포: 1이 2건, 2가 2건, 3이 3건, NULL이 3건) 따라서 결과는 A=10, B=7이다.

선택지별 풀이

① 7, 7은 COUNT(*)도 NULL 행을 제외한다고 착각한 결과이다.

② 10, 10은 COUNT(DEPT_ID)가 NULL 행까지 센다고 착각한 결과이다.

③ 7, 10은 두 함수의 동작을 서로 뒤바꾼 결과이다.

> **보충 개념** COUNT(*)를 제외한 집계 함수(COUNT(컬럼), SUM, AVG, MIN, MAX)는 모두 NULL을 제외하고 계산한다. COUNT(DISTINCT DEPT_ID)였다면 NULL을 제외한 고유값 1, 2, 3만 세어 3이 된다.

13 2과목

다음 SQL 실행 결과로 가장 적절한 것은?

EMP

EMP_NAME	DEPT_ID
KING	20
ALLEN	10
JONES	NULL
CLARK	NULL

DEPT

DEPT_ID	DEPT_NAME	LOCATION
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO

```
SELECT E.EMP_NAME
FROM EMP E, DEPT D
WHERE E.DEPT_ID = D.DEPT_ID(+)
AND D.LOCATION <> 'DALLAS';
```

① ALLEN, JONES, CLARK

② KING, ALLEN, CLARK

③ KING, ALLEN

④ **ALLEN**

정답 ④

핵심 해설

정답은 ④ ALLEN이다. E.DEPT_ID = D.DEPT_ID(+)는 EMP를 기준으로 한 LEFT OUTER JOIN이지만, 그 뒤 D.LOCATION <> 'DALLAS' 조건에 (+)가 붙지 않아 조인 후 필터로 동작한다. 1단계(아우터 조인 결과):

EMP_NAME	D.DEPT_ID	D.LOCATION
KING	20	DALLAS
ALLEN	10	NEW YORK
JONES	NULL	NULL
CLARK	NULL	NULL

2단계(D.LOCATION <> 'DALLAS' 필터): KING은 DALLAS라 제외, ALLEN은 NEW YORK이라 통과, JONES·CLARK는 NULL <> 'DALLAS'가 UNKNOWN이 되어 제외된다. 결과는 ALLEN 1건이다.

선택지별 풀이

① ALLEN, JONES, CLARK는 NULL <> 'DALLAS'를 참으로 잘못 판단해 KING만 제외한 결과이다. 참고로 조건에도 (+)를 붙여 D.LOCATION(+) <> 'DALLAS'로 쓰면 KING도 DEPT 쪽이 NULL로 채워진 채 남아 4명 모두 출력된다.

② KING, ALLEN, CLARK는 조인·필터 어느 쪽 규칙과도 맞지 않는다.

③ KING, ALLEN은 아우터 조인 없이 일반 조인만 하고 DALLAS 필터를 무시한 결과이다.

> **보충 개념** Oracle (+) 아우터 조인에서 Inner 쪽 테이블 조건에 (+)를 빼면 NULL 행이 걸러져 사실상 INNER JOIN처럼 동작한다. ANSI 문법에서는 조건을 ON 절에 두느냐 WHERE 절에 두느냐의 차이와 같다.

14 2과목

다음 SQL의 결과 건수로 맞는 것을 고르시오.

TBL

COL
100
200
300

```
SELECT *
FROM TBL
WHERE COL IN (100, 200, 100);
```

① 1　② **2**　③ 3　④ 오류가 발생한다.

정답 ②

핵심 해설

정답은 ② 2이다. IN 연산자는 괄호 안 값 목록 중 하나와 일치하면 참이 되는 OR 조건의 축약형이다. COL IN (100, 200, 100)은 COL = 100 OR COL = 200 OR COL = 100과 같다. 1단계: COL = 100인 행 1건(100)이 참이다. 2단계: COL = 200인 행 1건(200)이 참이다. 3단계: COL = 300인 행은 목록에 없으므로 제외된다. 목록에 100이 두 번 있어도 각 행은 조건을 만족하는지 여부만 판단되므로 결과 행이 중복 출력되지 않는다. 결과는 100, 200의 2건이다.

선택지별 풀이

① 1은 중복된 100 때문에 200이 무시된다고 오해한 결과이다.

③ 3은 목록 값 개수(3개)만큼 행이 나온다고 착각한 결과이다.

④ IN 목록에 같은 값이 중복되어도 문법 오류가 아니며 정상 실행된다.

> **보충 개념** IN 목록에 NULL이 포함되어도 COL = NULL은 UNKNOWN이라 무시될 뿐 오류는 없다. 반대로 NOT IN 목록(서브쿼리 포함)에 NULL이 하나라도 있으면 결과가 0건이 되는 점이 자주 출제된다.

---

15 2과목

다음 SQL 실행 결과로 가장 적절한 것은?

TBL

A	B
10	NULL
10	NULL
20	NULL
20	30
20	40
30	NULL

```
SELECT (COUNT(*) + COUNT(B)) AS CNT
FROM TBL
GROUP BY A
ORDER BY A;
```

① NULL, 5, NULL　② **2, 5, 1**

③ 8　④ 5

정답 ②

핵심 해설

정답은 ② 2, 5, 1이다. GROUP BY A로 A값별 그룹을 만든 뒤, 그룹마다 COUNT(*)(전체 행 수)와 COUNT(B)(B가 NULL이 아닌 행 수)를 더한다.

A	COUNT(*)	COUNT(B)	CNT
10	2	0	2
20	3	2	5
30	1	0	1

A=10 그룹은 B가 모두 NULL이라 COUNT(B)=0이고, A=20 그룹은 B가 30, 40인 2건만 세어진다. ORDER BY A로 정렬하면 2, 5, 1 순서로 3행이 출력된다.

선택지별 풀이

① NULL, 5, NULL은 COUNT(B)가 NULL을 반환한다고 착각한 결과이다. COUNT는 대상이 없어도 NULL이 아닌 0을 반환하므로 덧셈 결과도 NULL이 되지 않는다.

③ 8은 GROUP BY 없이 전체 6건+B 2건을 계산한 결과이다.

④ 5는 A=20 그룹 하나만 출력된다고 본 결과이다.

> **보충 개념** SUM(B)처럼 대상 값이 모두 NULL인 그룹에서는 SUM이 NULL을 반환하므로 COUNT(*) + SUM(B)였다면 A=10, 30 그룹 결과가 NULL이 된다. COUNT만 0을 반환한다는 차이를 기억해야 한다.

16 2과목

다음 SQL 실행 결과로 가장 적절한 것은?

TBL1

COL1
10
12
15
22
28
30
31

TBL2

COL2
15
30

```
SELECT COL1
FROM TBL1
WHERE COL1 <= ALL (SELECT COL2 FROM TBL2);
```

① 10, 12, 15

② 15, 22, 28, 30

③ 10, 12, 15, 22, 28, 30

④ 10, 12, 15, 22, 28, 30, 31

정답 ①

핵심 해설

정답은 ① 10, 12, 15이다. COL1 <= ALL (서브쿼리)는 서브쿼리가 반환한 모든 값보다 작거나 같아야 참이므로, 사실상 서브쿼리 결과의 최솟값 이하인 행을 찾는 조건이다. 1단계: 서브쿼리 결과는 COL2 = 15, 30이다. 2단계: COL1 <= 15 AND COL1 <= 30을 동시에 만족해야 하므로 COL1 <= 15와 같다. 3단계: TBL1에서 15 이하인 값은 10, 12, 15이다. 22, 28, 30, 31은 15보다 커서 제외된다.

선택지별 풀이

② 15, 22, 28, 30은 15 이상 30 이하, 즉 범위 조건(BETWEEN)으로 잘못 해석한 결과이다.

③ 10, 12, 15, 22, 28, 30은 <= ANY(최댓값 30 이하)로 해석한 결과이다.

④ 전체 행은 조건이 무시된다고 본 결과이다.

> **보충 개념** < ALL은 최솟값보다 작음, > ALL은 최댓값보다 큼, < ANY는 최댓값보다 작음, > ANY는 최솟값보다 큼과 같다. 서브쿼리 결과가 0건이면 ALL 조건은 항상 참이 되어 전체 행이 반환된다는 점도 함께 기억한다.

17 2과목

다음 중 순수 관계 연산자에 대한 설명으로 옳지 않은 것은?

① 관계형 데이터베이스에서 테이블(릴레이션)의 데이터를 처리하기 위해 고안된 관계 연산자이다.

② PROJECT - 특정 속성(컬럼)만 추출하는 연산자이다.

③ DIVIDE - 특정 조건을 모두 만족하는 대상을 찾는 연산이며 SQL에서는 직접 지원하지 않아 NOT EXISTS 등으로 표현한다.

④ SELECT - SQL의 SELECT 절에 해당한다.

정답 ④

핵심 해설

정답은 ④ SELECT - SQL의 SELECT 절에 해당한다이다. 순수 관계 연산자의 SELECT는 릴레이션에서 조건을 만족하는 튜플(행)을 골라내는 연산이므로 SQL에서는 WHERE 절에 해당한다. SQL의 SELECT 절처럼 특정 컬럼을 고르는 연산은 PROJECT이다.

선택지별 풀이

① 순수 관계 연산자(SELECT, PROJECT, JOIN, DIVIDE)는 관계형 데이터베이스의 릴레이션을 처리하기 위해 새로 고안된 연산자이므로 옳은 설명이다.

② PROJECT는 릴레이션에서 원하는 속성(컬럼)만 추출하는 연산으로, SQL의 SELECT 절(컬럼 목록)에 해당하므로 옳다.

③ DIVIDE는 나누는 릴레이션의 값을 모두 가진 튜플을 찾는 연산이며, 현재 SQL에 직접 대응하는 구문이 없어 NOT EXISTS 이중 부정 등으로 구현하므로 옳다.

> **보충 개념** 대응 관계는 SELECT→WHERE 절, PROJECT→SELECT 절(컬럼), JOIN→FROM·WHERE 또는 JOIN 절, DIVIDE→직접 지원 없음이다. 일반 집합 연산자(UNION, INTERSECTION, DIFFERENCE, PRODUCT)는 각각 UNION, INTERSECT, MINUS/EXCEPT, CROSS JOIN에 대응한다.

18 2과목

다음 TBL의 COL 값 중 LIKE '%A_C%' 패턴에 매칭되는 문자열은 몇 건인가?

TBL

COL
A1C
A-CD
A_C
AB_C
ABC

① 2개 ② 3개 **③ 4개** ④ 5개

정답 ③

핵심 해설

정답은 ③ 4개이다. LIKE 패턴에서 %는 0개 이상의 임의 문자, _는 정확히 1개의 임의 문자를 뜻한다. ESCAPE를 지정하지 않았으므로 '%A_C%'의 _도 와일드카드이며, 'A 다음 임의 1문자 다음 C'가 문자열 어딘가에 있으면 매칭된다.

COL	A?C 부분 문자열	매칭
A1C	A1C	O
A-CD	A-C	O

COL	A?C 부분 문자열	매칭
A_C	A_C	O
AB_C	없음(A 뒤 두 번째 문자가 _)	X
ABC	ABC	O

AB_C는 A 바로 다음 문자가 B, 그 다음이 _이므로 A와 C 사이에 문자가 2개 있어 패턴에 맞지 않는다. 따라서 매칭 건수는 4건이다.

선택지별 풀이

① 2개는 _를 밑줄 문자 자체로 읽어 A_C만 세거나, 기호(-)가 들어간 A-CD를 빠뜨린 결과이다. _는 문자 종류와 무관하게 정확히 1문자와 매칭된다.

② 3개는 뒤에 D가 붙은 A-CD를 제외한 결과이다. 패턴 뒤쪽의 %가 0개 이상의 문자를 허용하므로 A-CD도 매칭된다.

④ 5개는 %가 A와 C 사이에도 들어간다고 보아 AB_C까지 포함한 결과이다.

> **보충 개념** 밑줄 문자 자체를 찾으려면 LIKE '%A_C%' ESCAPE '\'처럼 ESCAPE 문자를 지정해야 한다. 그 경우에는 A_C 1건만 매칭된다.

---

19 2과목

다음 SQL 실행 결과로 가장 적절한 것은?

SALES

ID	AMOUNT
1000	40000
2000	30000
3000	35000
4000	45000
5000	50000

```
SELECT ID
FROM SALES
ORDER BY AMOUNT DESC
OFFSET 2 ROWS FETCH NEXT 1 ROWS ONLY;
```

① 1000
② 5000, 4000, 1000
③ 2000, 3000, 1000
④ 5000

정답 ①

핵심 해설

정답은 ① 1000이다. OFFSET n ROWS는 정렬된 결과에서 앞의 n행을 건너뛰고, FETCH NEXT m ROWS ONLY는 그 다음 m행만 반환한다. 1단계(ORDER BY AMOUNT DESC 정렬):

순서	ID	AMOUNT
1	5000	50000
2	4000	45000
3	1000	40000
4	3000	35000
5	2000	30000

2단계: OFFSET 2 ROWS로 1·2번째 행(5000, 4000)을 건너뛴다. 3단계: FETCH NEXT 1 ROWS ONLY로 3번째 행 1건만 반환하므로 결과는 ID=1000이다.

선택지별 풀이

② 5000, 4000, 1000은 OFFSET을 무시하고 상위 3건을 가져온 결과이다.

③ 2000, 3000, 1000은 오름차순으로 정렬해 3건을 가져온 결과이다.

④ 5000은 OFFSET 없이 1위 행만 가져온 결과이다.

> **보충 개념** OFFSET은 0부터 건너뛸 행 수를 세므로 OFFSET 2는 3번째 행부터 시작한다. 동순위 값이 있을 때 함께 가져오려면 FETCH NEXT n ROWS WITH TIES를 사용한다.

20 2과목

다음 SQL과 동일한 결과를 출력하는 SQL은 무엇인가?

```
SELECT 주문일자, 상품코드, COUNT(*)
FROM 주문
GROUP BY ROLLUP((주문일자, 상품코드));
```

① SELECT 주문일자, 상품코드, COUNT(*) FROM 주문 GROUP BY 주문일자, ROLLUP(상품코드);

② SELECT 주문일자, 상품코드, COUNT(*) FROM 주문 GROUP BY ROLLUP(주문일자, 상품코드);

③ SELECT DISTINCT 주문일자, 상품코드, COUNT(*) FROM 주문 GROUP BY ROLLUP((주문일자, 상품코드), NULL);

④ SELECT 주문일자, 상품코드, COUNT(*) FROM 주문 GROUP BY GROUPING SETS((주문일자, 상품코드));

정답 ③

핵심 해설

정답은 ③ SELECT DISTINCT … GROUP BY ROLLUP((주문일자, 상품코드), NULL)이다. ROLLUP((주문일자, 상품코드))처럼 괄호로 묶은 컬럼은 하나의 단위로 취급되어, 그룹핑 단위가 (주문일자, 상품코드)와 ()(전체 합계) 두 가지만 생성된다.

선택지별 풀이

③ 의 ROLLUP((주문일자, 상품코드), NULL)은 ((주문일자, 상품코드), NULL), (주문일자, 상품코드), () 세 단위를 만든다. 상수 NULL로 그룹핑해도 그룹이 더 나뉘지 않으므로 앞의 두 단위는 같은 상세 행을 중복 생성한다. DISTINCT가 이 중복을 제거하므로 결과는 상세 행 + 전체 합계 1행으로 원래 SQL과 같다.

① 주문일자, ROLLUP(상품코드)는 (주문일자, 상품코드)와 (주문일자) 단위를 만들어 일자별 소계는 있지만 전체 합계가 없다.

② ROLLUP(주문일자, 상품코드)는 (주문일자, 상품코드), (주문일자), () 세 단위를 만들어 일자별 소계가 추가로 출력된다.

④ GROUPING SETS((주문일자, 상품코드))는 상세 그룹 하나만 만들어 일반 GROUP BY 주문일자, 상품코드와 같고 전체 합계가 없다.

> **보충 개념** ROLLUP(A, B)는 (A, B), (A), () 순으로 계층적 소계를 만들고, 괄호로 묶은 (A, B)는 한 컬럼처럼 취급된다. 원래 SQL은 GROUPING SETS((주문일자, 상품코드), ())와도 같은 결과이다.

21 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

```
SELECT SUBSTR('abcdefg', LENGTH('abcdefg') - 3) FROM DUAL;
```

① abcd ② **defg** ③ d ④ abc

정답 ②

핵심 해설

정답은 ② defg이다. SUBSTR(문자열, 시작위치)처럼 길이 인자를 생략하면 시작위치부터 문자열 끝까지 잘라 낸다. 계산 과정은 다음과 같다. 1단계: LENGTH('abcdefg') = 7 2단계: 시작위치 = 7 - 3 = 4 3단계: SUBSTR('abcdefg', 4) → 4번째 문자 d부터 끝까지 → 'defg'

선택지별 풀이

① abcd는 SUBSTR('abcdefg', 1, 4)처럼 첫 글자부터 4글자를 자른 결과로, 시작위치를 1로 착각한 경우이다.

③ d는 SUBSTR('abcdefg', 4, 1)처럼 길이 1을 지정했을 때의 결과이며, 이 문제는 길이 인자가 없으므로 끝까지 반환된다.

④ abc는 앞의 3글자로, '-3'을 잘라 낼 글자 수로 오해한 경우이다.

보충 개념 Oracle의 SUBSTR은 위치가 1부터 시작하며, 시작위치에 음수를 주면 뒤에서부터 센다. 예를 들어 SUBSTR('abcdefg', -4)도 'defg'를 반환한다.

22 2과목

다음 SQL의 빈칸에 알맞은 함수는 무엇인가?

TBL

NAME	SCORE
KIM	90
LEE	90
PARK	85
CHOI	80

```
SELECT NAME, SCORE,
______() OVER (ORDER BY SCORE DESC) AS RNK
FROM TBL;
```

실행 결과

NAME	SCORE	RNK
KIM	90	1
LEE	90	1
PARK	85	3
CHOI	80	4

① RANK ② DENSE_RANK ③ ROW_NUMBER ④ ROWNUM

정답 ①

핵심 해설

정답은 ① RANK이다. 실행 결과에서 SCORE 90인 KIM과 LEE가 공동 1위이고, 그다음 85인 PARK이 2가 아니라 3을 받았다. 동일 값에 같은 순위를 주고 동순위 건수만큼 다음 순위를 건너뛰는 함수는 RANK이다.

NAME	SCORE	RANK	DENSE_RANK	ROW_NUMBER
KIM	90	1	1	1
LEE	90	1	1	2
PARK	85	3	2	3
CHOI	80	4	3	4

선택지별 풀이

② DENSE_RANK는 동순위 다음 순위를 건너뛰지 않으므로 PARK=2, CHOI=3이 되어 실행 결과와 다르다.

③ ROW_NUMBER는 같은 점수라도 고유한 순번을 부여하므로 KIM과 LEE가 1, 2로 나뉜다.

④ ROWNUM은 Oracle의 의사컬럼(pseudo column)으로 윈도우 함수가 아니어서 ROWNUM() OVER (...) 형태로 쓸 수 없고 오류가 발생한다.

보충 개념 순위 함수 3종은 동점 처리 방식만 다르다. RANK는 1,1,3 / DENSE_RANK는 1,1,2 / ROW_NUMBER는 1,2,3으로 기억하면 된다.

23 2과목

다음 중 테이블명으로 가장 적절한 것은 무엇인가?

① 201504table_TB ② table

③ test-n202605 **④ tbl_test260531#**

정답 ④

핵심 해설

정답은 ④ tbl_test260531#이다. Oracle 테이블명 규칙은 문자로 시작하고, 문자·숫자와 특수문자 , $, #만 사용할 수 있으며, 예약어는 쓸 수 없고 30바이트(12.2 이후 128바이트) 이내여야 한다. tbl_test260531#은 문자 t로 시작하고 영문·숫자··#만 사용했으므로 규칙을 모두 만족한다.

선택지별 풀이

① 201504table_TB는 숫자로 시작하므로 테이블명으로 사용할 수 없다.

② table은 SQL 예약어(CREATE TABLE 등에 쓰이는 키워드)이므로 테이블명으로 사용할 수 없다.

③ test-n202605는 허용되지 않는 특수문자 '-'(하이픈)가 포함되어 있어 사용할 수 없다. 하이픈은 빼기 연산자로 해석된다.

> **보충 개념** 큰따옴표로 감싸면("201504table_TB") 규칙을 벗어난 이름도 만들 수 있지만, 시험에서는 따옴표 없는 일반 식별자 규칙을 기준으로 판단한다. 허용 특수문자는 _, $, # 세 가지만 기억하자.

24 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

EMP

EMP_NAME	DEPT_ID	SAL
SMITH	10	100
ALLEN	10	200
WARD	20	300
JONES	30	400
MARTIN	30	100

```
SELECT *
FROM (SELECT DEPT_ID, SAL FROM EMP)
PIVOT (SUM(SAL) FOR DEPT_ID IN (10 AS A, 20 AS B, 30 AS C));
```

① 컬럼명 10, 20, 30 / 값 300, 300, 500

② 컬럼명 10, 20, 30 / 값 200, 300, 400

③ 컬럼명 A, B, C / 값 300, 300, 500

④ 컬럼명 A, B, C / 값 200, 300, 400

정답 ③

핵심 해설

정답은 ③ 컬럼명 A, B, C / 값 300, 300, 500이다. PIVOT은 FOR 절 컬럼(DEPT_ID)의 값을 컬럼으로 펼치고, IN 목록에 별칭을 주면 그 별칭이 결과 컬럼명이 된다. 인라인 뷰에는 DEPT_ID와 SAL만 있으므로 나머지 컬럼이 없어 전체가 1행으로 집계된다. 1단계: DEPT_ID=10 → SMITH 100 + ALLEN 200 = 300 → 컬럼 A 2단계: DEPT_ID=20 → WARD 300 = 300 → 컬럼 B 3단계: DEPT_ID=30 → JONES 400 + MARTIN 100 = 500 → 컬럼 C

A	B	C
300	300	500

선택지별 풀이

① 값은 맞지만, IN 목록에 AS A, AS B, AS C 별칭을 지정했으므로 컬럼명이 10, 20, 30이 아니다.

② 컬럼명도 별칭을 무시했고, 값도 SUM이 아니라 부서별 MAX(200, 300, 400)를 구한 것처럼 틀렸다.

④ 컬럼명은 맞지만 값이 SUM이 아닌 MAX 기준이라 틀렸다.

보충 개념 PIVOT에서 인라인 뷰에 남은 컬럼(집계 대상·FOR 컬럼 제외)이 암묵적 GROUP BY 기준이 된다. 그래서 EMP_NAME 같은 컬럼을 인라인 뷰에서 미리 제거해야 원하는 1행 결과가 나온다.

25 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

TBL1

COL	COL1
2	NULL
3	NULL
4	40
5	40

TBL2

COL	COL2
1	30
2	40
3	40
4	NULL

```
SELECT AVG(A.COL1), AVG(NVL(B.COL2, 0))
FROM TBL1 A
LEFT OUTER JOIN TBL2 B
ON    A.COL = B.COL;
```

① 20, 20 ② **40, 20** ③ 40, 40 ④ 20, 40

정답 ②

핵심 해설

정답은 ② 40, 20이다. LEFT OUTER JOIN은 TBL1(A)의 모든 행을 유지하고, 매칭되는 TBL2(B) 행이 없으면 B 컬럼을 NULL로 채운다. 조인 결과는 다음과 같다.

A.COL	A.COL1	B.COL	B.COL2	NVL(B.COL2,0)
2	NULL	2	40	40
3	NULL	3	40	40
4	40	4	NULL	0
5	40	NULL	NULL	0

1단계: AVG(A.COL1) → NULL은 제외하고 (40 + 40) / 2 = 40 2단계: AVG(NVL(B.COL2, 0)) → NULL이 0으로 바뀌어 4행 모두 포함, (40 + 40 + 0 + 0) / 4 = 20① 20, 20은 AVG(A.COL1)에서 NULL 행까지 분모에 넣어 80 / 4로 계산한 오답이다.
③ 40, 40은 두 번째 식에서 NVL을 무시하고 NULL을 제외해 80 / 2로 계산한 오답이다.
④ 20, 40은 두 값의 NULL 처리 방식을 서로 뒤바꾼 오답이다.

보충 개념 집계 함수는 NULL을 제외하고 계산하므로 AVG의 분모가 달라진다. NVL로 NULL을 0으로 바꾸면 해당 행이 분모에 포함되어 평균이 낮아진다. TBL2의 COL=1 행은 LEFT JOIN 기준 테이블(TBL1)에 없어 결과에 나타나지 않는다.

26 2과목

다음 중 계층형 질의에 대한 설명으로 옳지 않은 것은?

① `CONNECT BY` 절은 부모 데이터를 기준으로 자식 데이터를 지정하여 계층 구조를 연결하는 역할을 한다.

② PRIOR 키워드는 CONNECT BY 절에서 부모-자식 컬럼의 방향을 지정하는 데 사용된다.

③ NOCYCLE 옵션을 사용하면, 순환참조가 발생한 이후의 데이터까지 포함하여 결과가 출력된다.

④ WHERE 절의 조건은 START WITH와 CONNECT BY로 계층구조가 모두 생성된 뒤에 적용되어, 그중 조건을 만족하는 행만 걸러서 보여준다.

정답 ③

핵심 해설

정답은 ③ NOCYCLE이 순환참조 이후의 데이터까지 출력한다는 설명이다. NOCYCLE은 계층 전개 중 이미 방문한 데이터가 다시 나타나는 순환(Cycle)을 만났을 때 오류를 내지 않게 하는 옵션이다. 다만 순환이 발생한 지점에서 전개를 멈추므로, 순환참조가 발생한 이후의 데이터는 출력하지 않는다. 따라서 "이후의 데이터까지 포함하여 출력된다"는 설명은 옳지 않다.

선택지별 풀이

① CONNECT BY 절은 다음에 전개될 자식 데이터를 지정하여 부모와 자식을 연결하는 구문이므로 옳은 설명이다.

② PRIOR는 CONNECT BY 절에서 어느 쪽이 부모 컬럼인지 지정한다. PRIOR 자식 = 부모 형태면 순방향(부모→자식), PRIOR 부모 = 자식 형태면 역방향(자식→부모)으로 전개되므로 옳은 설명이다.

④ WHERE 절은 START WITH와 CONNECT BY로 계층 전개를 모두 마친 뒤 필터 조건으로 적용되어 조건을 만족하는 행만 남기므로 옳은 설명이다.

> **보충 개념** NOCYCLE 없이 순환이 있으면 ORA-01436 오류가 발생한다. 또한 CONNECT BY 절에 쓴 조건은 전개 자체를 제한하고, WHERE 절 조건은 전개 후 결과를 거르는 점이 자주 비교되어 출제된다.

---

27 2과목

다음 조건을 만족하는 데이터를 찾는 WHERE 절 조건으로 가장 적절하지 않은 것은?

조건

소속팀(TEAM)이 '삼성블루윙즈' 또는 '전남드래곤즈'이고, 포지션(POSITION)이 'MF'이며, 키(HEIGHT)가 175 이상 180 이하인 선수

① (TEAM = '삼성블루윙즈' OR TEAM = '전남드래곤즈') AND POSITION = 'MF' AND HEIGHT >= 175 AND HEIGHT <= 180

② TEAM = '삼성블루윙즈' OR TEAM = '전남드래곤즈' AND POSITION = 'MF' AND HEIGHT >= 175 AND HEIGHT <= 180

③ POSITION = 'MF' AND HEIGHT BETWEEN 175 AND 180 AND (TEAM = '삼성블루윙즈' OR TEAM = '전남드래곤즈')

④ POSITION = 'MF' AND HEIGHT >= 175 AND HEIGHT <= 180 AND (TEAM = '삼성블루윙즈' OR TEAM = '전남드래곤즈')

정답 ②

핵심 해설

정답은 ② 괄호 없이 OR와 AND를 섞어 쓴 조건이다. SQL의 논리 연산자 우선순위는 NOT > AND > OR이므로 괄호가 없으면 AND가 먼저 결합된다.

선택지별 풀이

② 는 다음과 같이 해석된다. TEAM = '삼성블루윙즈' OR (TEAM = '전남드래곤즈' AND POSITION = 'MF' AND HEIGHT >= 175 AND HEIGHT <= 180) 즉 삼성블루윙즈 소속이면 포지션과 키에 관계없이 모두 조회되므로 문제의 조건과 다르다.

① 팀 조건을 괄호로 묶어 OR를 먼저 처리한 뒤 포지션과 키 조건을 AND로 결합했으므로 조건을 정확히 표현한다.

③ HEIGHT BETWEEN 175 AND 180은 HEIGHT >= 175 AND HEIGHT <= 180과 같고(양 끝 포함), 팀 조건도 괄호로 묶었으므로 적절하다.

④ ①과 조건의 순서만 다를 뿐 AND로만 연결되어 있고 팀 조건이 괄호로 묶여 있으므로 동일한 결과를 반환한다.

> **보충 개념** AND로만 연결된 조건은 순서를 바꿔도 결과가 같다. OR가 AND와 섞일 때는 반드시 괄호로 의도한 결합 순서를 명시해야 한다.

---

28 2과목

다음 중 "연봉(SAL)이 3,000을 초과하면서, 상위 직급자(관리자, MGR)가 존재하는 사원"을 찾는 SQL로 가장 적절하지 않은 것은? (단, EMP_ID는 EMP 테이블의 기본키이다.)

① `SELECT EMP_ID, EMP_NAME, SAL FROM EMP E WHERE SAL > 3000 AND EXISTS (SELECT 1 FROM EMP M WHERE M.EMP_ID = E.MGR_ID);`

② `SELECT EMP_ID, EMP_NAME, SAL FROM EMP WHERE SAL > 3000 AND MGR_ID IN (SELECT EMP_ID FROM EMP);`

③ `SELECT EMP_ID, EMP_NAME, SAL FROM EMP E WHERE SAL > 3000 AND NOT EXISTS (SELECT 1 FROM EMP M WHERE M.EMP_ID <> E.MGR_ID);`

④ `SELECT E1.EMP_ID, E1.EMP_NAME, E1.SAL FROM EMP E1, EMP E2 WHERE E1.MGR_ID = E2.EMP_ID AND E1.SAL > 3000;`

정답 ③

핵심 해설

정답은 ③ NOT EXISTS 안에서 <>로 비교한 SQL이다. NOT EXISTS (SELECT 1 FROM EMP M WHERE M.EMP_ID <> E.MGR_ID)는 "관리자 사번과 다른 사번을 가진 사원이 한 명도 없을 때" 참이 된다. 사원이 여러 명이면 MGR_ID와 다른 EMP_ID는 거의 항상 존재하므로 관리자가 있는 사원이 오히려 걸러진다. 반대로 MGR_ID가 NULL인 사원은 M.EMP_ID <> NULL이 모두 UNKNOWN이 되어 서브쿼리가 0건이므로 NOT EXISTS가 참이 되어, 관리자가 없는 사원이 조회된다. 의도와 정반대이다.

선택지별 풀이

① 상관 서브쿼리로 E.MGR_ID와 같은 EMP_ID를 가진 행이 존재하는지 확인하므로 관리자가 있는 사원을 정확히 찾는다.

② MGR_ID가 EMP 테이블의 EMP_ID 목록에 포함되는지 IN으로 확인한다. MGR_ID가 NULL이면 IN 결과가 UNKNOWN이라 제외되므로 적절하다.

④ 자기 조인(Self Join)으로 E1의 MGR_ID와 E2의 EMP_ID를 연결한다. EMP_ID가 기본키라 E1 한 행당 E2가 최대 1행만 매칭되어 중복도 생기지 않으므로 적절하다.

> **보충 개념** "관리자가 존재한다"는 EXISTS(M.EMP_ID = E.MGR_ID)로 표현하며, 이를 부정형으로 바꾸려면 NOT EXISTS의 조건을 <>로 뒤집는 것이 아니라 EXISTS 자체를 유지해야 한다. 서브쿼리 조건의 부정과 EXISTS의 부정은 의미가 다르다.

29 2과목

다음 SQL의 실행 결과로 옳은 것은?

```
SELECT SALE_DATE, AMT,
SUM(AMT) OVER (ORDER BY SALE_DATE) AS CUM_AMT
FROM SALES;
```

① 날짜순으로 정렬된 각 행까지의 누적 합계를 계산한다.

② 전체 금액의 합계가 모든 행에 동일하게 표시된다.

③ 날짜별로 그룹화되어 금액의 합계가 표시된다.

④ 오류가 발생한다. (OVER 절에는 PARTITION BY가 반드시 필요하다)

정답 ①

핵심 해설

정답은 ① 날짜순 각 행까지의 누적 합계를 계산한다는 설명이다. 윈도우 함수에 ORDER BY만 지정하고 윈도우 절을 생략하면 기본값 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW가 적용된다. 따라서 SALE_DATE 순으로 첫 행부터 현재 행까지의 AMT를 더한 누적 합계가 CUM_AMT로 표시된다. 예) 1일 AMT=100, 2일 AMT=200, 3일 AMT=50이면 CUM_AMT는 100, 300, 350이 된다.

선택지별 풀이

② 모든 행에 전체 합계가 동일하게 표시되는 것은 SUM(AMT) OVER ()처럼 ORDER BY 없이 사용했을 때의 결과이다.

③ 날짜별로 그룹화하여 합계를 1행씩 보여 주는 것은 GROUP BY SALE_DATE의 동작이다. 윈도우 함수는 행 수를 줄이지 않고 원래 행을 모두 유지한다.

④ OVER 절의 PARTITION BY는 선택 사항이다. 생략하면 전체 행을 하나의 파티션으로 보고 계산하므로 오류가 발생하지 않는다.

보충 개념 기본 윈도우가 RANGE 기준이므로 같은 SALE_DATE를 가진 행이 여러 개면 그 행들은 동일한 누적값(해당 날짜까지의 합)을 갖는다. 행 단위로 누적하려면 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW를 명시한다.

30 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

```
SELECT *
FROM (SELECT BOARD_TYPE, TITLE,
ROW_NUMBER() OVER (PARTITION BY BOARD_TYPE
ORDER BY REG_DATE DESC) AS RN
FROM BOARD)
WHERE RN = 1;
```

① 게시판 유형(BOARD_TYPE)별로 가장 최근 글 1건씩만 출력된다.

② 전체 게시글 중 가장 최근 글 1건만 출력된다.

③ 게시판 유형과 무관하게 상위 2건이 출력된다.

④ 게시판 유형별 게시글 건수가 출력된다.

정답 ①

핵심 해설

정답은 ① 게시판 유형별로 가장 최근 글 1건씩만 출력된다는 설명이다. ROW_NUMBER() OVER (PARTITION BY BOARD_TYPE ORDER BY REG_DATE DESC)는 게시판 유형별로 파티션을 나눈 뒤, 각 파티션 안에서 등록일이 최신인 글부터 1, 2, 3… 순번을 매긴다. 바깥 쿼리에서 RN = 1만 남기므로 게시판 유형마다 가장 최근 글 1건씩만 출력된다. 예) BOARD_TYPE이 공지·자유 두 가지라면 공지의 최신 글 1건, 자유의 최신 글 1건으로 총 2행이 출력된다.

선택지별 풀이

② 전체 게시글 중 최신 글 1건만 출력되려면 PARTITION BY 없이 ORDER BY REG_DATE DESC만 지정해야 한다.

③ PARTITION BY BOARD_TYPE으로 유형별로 순번을 다시 매기므로 유형과 무관하다는 설명이 틀렸고, RN = 1 조건이므로 상위 2건이 아니라 유형별 1건이다.

④ 유형별 건수를 구하려면 COUNT(*)와 GROUP BY BOARD_TYPE을 사용해야 한다. 이 쿼리는 BOARD_TYPE, TITLE, RN 컬럼을 반환한다.

보충 개념 윈도우 함수는 WHERE 절에서 직접 사용할 수 없으므로 인라인 뷰로 감싼 뒤 바깥에서 RN을 필터링한다. 같은 REG_DATE에 동점이 있으면 ROW_NUMBER는 그중 하나만 1로 주므로, 동점을 모두 남기려면 RANK를 사용한다.

31 2과목

다음 SQL의 결과로 가장 적절한 것은?

PRODUCT

PROD_ID	PROD_NAME	PARENT_ID
100	노트북	300
200	마우스	400
300	전자제품	NULL
400	액세서리	500
500	잡화	NULL

```
SELECT *
FROM PRODUCT
WHERE PROD_ID NOT IN (SELECT PARENT_ID FROM PRODUCT);
```

① PRODUCT 테이블의 모든 행

② 상위 카테고리로 지정되지 않은 상품의 행

③ 공집합 ④ 오류 발생

정답 ③

핵심 해설

정답은 ③ 공집합이다. 서브쿼리 SELECT PARENT_ID FROM PRODUCT의 결과는 300, 400, NULL, 500, NULL로 NULL이 포함된다. NOT IN (300, 400, NULL, 500, NULL)은 PROD_ID <> 300 AND PROD_ID <> 400 AND PROD_ID <> NULL AND …로 전개되는데, PROD_ID <> NULL은 항상 UNKNOWN이므로 전체 조건은 FALSE 또는 UNKNOWN이 되고 TRUE가 되는 행이 없다.

PROD_ID	NULL 외 값과 비교	NULL과 비교	최종
100	TRUE	UNKNOWN	제외
200	TRUE	UNKNOWN	제외
300	FALSE(300 일치)	UNKNOWN	제외
400	FALSE(400 일치)	UNKNOWN	제외
500	FALSE(500 일치)	UNKNOWN	제외

선택지별 풀이

① 모든 행이 나오려면 모든 행에서 NOT IN 조건이 TRUE여야 하지만, NULL 비교 때문에 TRUE가 되는 행이 하나도 없다.

② 의도한 결과(100 노트북, 200 마우스)지만 서브쿼리에 NULL이 섞여 있어 나오지 않는다. 서브쿼리에 WHERE PARENT_ID IS NOT NULL을 추가해야 이 두 행이 반환된다.

④ 서브쿼리가 단일 컬럼을 반환하고 PROD_ID와 데이터 타입도 같으므로 문법 오류는 없다.

> **보충 개념** NOT IN 서브쿼리 결과에 NULL이 하나라도 있으면 결과는 공집합이 된다. NOT EXISTS로 바꾸면 NULL의 영향을 받지 않아 100, 200이 반환된다.

32 2과목

다음 SQL의 실행 결과에 대한 설명으로 가장 적절한 것은?

```
SELECT CEIL(7.1) AS A,
FLOOR(8.2) AS B,
ROUND(7.6) AS C,
TRUNC(8.9) AS D
FROM DUAL;
```

① A만 다른 값이 나온다. ② B와 D만 같은 값이 나온다.

③ 모든 함수의 결과는 같다. ④ 오류가 발생한다.

정답 ③

핵심 해설

정답은 ③ 모든 함수의 결과는 같다이다. 각 함수를 계산하면 다음과 같다.

컬럼	함수	동작	결과
A	CEIL(7.1)	크거나 같은 최소 정수(올림)	8
B	FLOOR(8.2)	작거나 같은 최대 정수(내림)	8
C	ROUND(7.6)	소수 첫째 자리에서 반올림	8
D	TRUNC(8.9)	소수점 이하 버림	8

A=8, B=8, C=8, D=8로 네 값이 모두 같다.

선택지별 풀이

① A도 7.1을 올림해 8이 되므로 다른 값이 아니다.

② B와 D뿐 아니라 A와 C도 8이므로 'B와 D만 같다'는 설명은 틀리다.

④ CEIL, FLOOR, ROUND, TRUNC는 모두 Oracle 숫자 함수로 DUAL에서 정상 실행된다. ROUND와 TRUNC는 자릿수 인자를 생략하면 0(정수 자리)으로 처리된다.

보충 개념 음수에서는 결과가 갈린다. CEIL(-7.1)=-7, FLOOR(-7.1)=-8, TRUNC(-7.1)=-7이므로 FLOOR와 TRUNC를 같은 함수로 혼동하지 않도록 한다.

33 2과목

다음 중 오류가 발생하지 않는 SQL을 모두 고른 것은?

TBL1

COL1	COL2	COL3
A	1	3
B	2	4

TBL2

COL1	COL2	COL3
A	C	1
B	D	2

```
-- 가
SELECT COL1, COL2, COL3 FROM TBL1
UNION
SELECT COL1, COL2 FROM TBL2;
-- 나
SELECT COL1, COL2, COL3 FROM TBL1 ORDER BY COL1
UNION
SELECT COL1, COL2, COL3 FROM TBL2;
-- 다
SELECT COL1, COL2 FROM TBL1
UNION
SELECT COL1, COL3 FROM TBL2 ORDER BY COL1;
```

① 가 ② **다** ③ 나, 다 ④ 모두 오류가 발생한다.

정답 ②

핵심 해설

정답은 ② 다이다. 집합 연산자는 위아래 SELECT의 컬럼 수와 대응 컬럼의 데이터 타입이 맞아야 하고, ORDER BY는 전체 집합 연산의 맨 마지막에 한 번만 쓸 수 있다. 가: 위 SELECT는 컬럼 3개, 아래 SELECT는 컬럼 2개로 컬럼 수가 달라 오류가 발생한다. 나: 첫 번째 SELECT 뒤에 ORDER BY를 쓰고 UNION을 이어 붙였으므로 오류가 발생한다. ORDER BY는 마지막 SELECT 뒤에만 올 수 있다. 다: TBL1(COL1 문자, COL2 숫자)과 TBL2(COL1 문자, COL3 숫자)는 컬럼 수 2개, 타입도 문자-문자, 숫자-숫자로 대응한다. 마지막 ORDER BY COL1도 첫 번째 SELECT의 컬럼명이므로 정상 실행되며, 결과는 중복이 제거된 (A, 1), (B, 2) 두 행이다.

선택지별 풀이

① 가는 컬럼 수 불일치로 오류가 발생하므로 정답이 될 수 없다.
③ 나는 ORDER BY 위치 때문에 오류가 발생하므로 포함될 수 없다.
④ 다는 정상 실행되므로 모두 오류가 발생하는 것은 아니다.

보충 개념 집합 연산 결과의 컬럼명은 첫 번째 SELECT 기준이며, 마지막 ORDER BY에는 첫 번째 SELECT의 컬럼명, 별칭 또는 위치 번호를 사용한다.

34 2과목

다음 SQL과 동일한 결과를 만드는 SQL을 모두 고른 것은? (단, TBL1.COL1에는 NULL과 중복 값이 없다.)

```
SELECT COL1 FROM TBL1
MINUS
```

```
SELECT COL2 FROM TBL2;
-- 가
SELECT COL1
FROM TBL1
WHERE COL1 NOT IN (SELECT COL2 FROM TBL2 WHERE COL2 IS NOT NULL);
-- 나
SELECT COL1
FROM TBL1
WHERE NOT EXISTS (SELECT 1 FROM TBL2 WHERE TBL1.COL1 = TBL2.COL2);
-- 다
SELECT T1.COL1
FROM TBL1 T1 LEFT OUTER JOIN TBL2 T2
ON T1.COL1 = T2.COL2
WHERE T2.COL2 IS NULL;
```

① 가 ② 가, 나 ③ 나, 다 **④ 가, 나, 다**

정답 ④

핵심 해설

정답은 ④ 가, 나, 다이다. MINUS는 TBL1.COL1 값 중 TBL2.COL2에 존재하지 않는 값을 중복 없이 반환한다. 단서에서 TBL1.COL1에 NULL과 중복이 없다고 했으므로 중복 제거와 NULL 처리 차이가 사라져, 세 가지 차집합 표현이 모두 같은 결과를 낸다. 가: NOT IN 서브쿼리에서 COL2 IS NOT NULL로 NULL을 미리 걸렀으므로, NOT IN이 NULL 때문에 공집합이 되는 문제가 없다. 동일하다. 나: NOT EXISTS는 TBL1.COL1과 같은 COL2가 하나도 없는 행만 남긴다. TBL2의 NULL은 어떤 값과도 일치하지 않으므로 영향을 주지 않는다. 동일하다. 다: LEFT OUTER JOIN 후 T2.COL2 IS NULL은 짝이 없는 TBL1 행만 남기는 안티 조인이다. TBL2에 같은 값이 여러 개 있어도 매칭된 행은 모두 제거되고, 매칭되지 않은 행은 한 번만 나오므로 중복도 생기지 않는다. 동일하다.

선택지별 풀이

① 나와 다도 같은 결과를 내므로 가만 고르면 부족하다.
② 다도 안티 조인으로 같은 결과를 내므로 빠지면 안 된다.
③ 가도 NULL을 제외해 NOT IN의 NULL 문제를 피했으므로 같은 결과다.

> **보충 개념** 단서가 없다면 MINUS는 중복을 제거하고 NULL끼리를 같은 값으로 보는 반면, NOT EXISTS나 안티 조인은 TBL1의 중복과 NULL 행을 그대로 남기므로 결과가 달라질 수 있다.

35 2과목

다음 SQL에 대한 설명으로 옳은 것은?

```
SELECT E.DEPT_ID, D.DEPT_ID
FROM EMP E, DEPT D;
```

① 두 테이블의 조인 조건이 없으므로 CROSS JOIN(Cartesian Product)이 발생한다.

② EMP와 DEPT가 DEPT_ID를 기준으로 자동 조인된다.

③ 오류가 발생하여 실행되지 않는다.

④ EQUI JOIN이 자동으로 적용된다.

정답 ①

핵심 해설

정답은 ① 조인 조건이 없으므로 CROSS JOIN(Cartesian Product)이 발생한다이다. FROM 절에 EMP와 DEPT를 콤마로 나열하고 WHERE 절에 조인 조건이 없으면 EMP의 모든 행과 DEPT의 모든 행이 하나씩 짝지어진다. 결과 행 수는 EMP 행 수 × DEPT 행 수이다(예: EMP 14건, DEPT 4건이면 56건).

선택지별 풀이

② 콤마 조인은 같은 이름의 컬럼이 있어도 자동으로 조인하지 않는다. 같은 이름 컬럼으로 자동 조인되는 것은 NATURAL JOIN을 명시했을 때뿐이다.

③ SELECT 절의 DEPT_ID를 E., D. 별칭으로 한정했으므로 컬럼 모호성 오류도 없고 정상 실행된다.

④ EQUI JOIN은 WHERE E.DEPT_ID = D.DEPT_ID처럼 등호 조인 조건을 직접 기술해야 적용된다.

> **보충 개념** N개 테이블을 조인하려면 최소 N 1개의 조인 조건이 필요하다. ANSI 표준에서는 CROSS JOIN 키워드로 카티션 곱을 명시적으로 표현한다.

36 2과목

다음 중 ROWNUM에 대한 설명으로 가장 적절한 것은?

① SELECT 절에서 결과 집합의 각 행에 순차적으로 부여되는 의사 컬럼(Pseudo Column)이다.

② WHERE 절 조건이 모두 평가된 이후 마지막에 한 번만 부여되는 고정값이다.

③ 인덱스 생성 시 자동으로 생성되는 물리적 컬럼이다.

④ 사용자가 직접 정의해야 하는 일반 컬럼이다.

정답 ①

핵심 해설

정답은 ① 결과 집합의 각 행에 순차적으로 부여되는 의사 컬럼이다. ROWNUM은 테이블에 저장된 컬럼이 아니라, 조회 시 행이 WHERE 조건을 통과해 결과 집합에 포함될 때마다 1, 2, 3 … 순으로 부여되는 Oracle의 의사 컬럼(Pseudo Column)이다. 그래서 WHERE ROWNUM <= 3처럼 상위 N건 추출에 사용한다.

선택지별 풀이

② ROWNUM은 WHERE 절 평가 과정에서 조건을 통과한 행에 차례로 부여된다. 모든 평가가 끝난 뒤 한 번에 붙는 고정값이 아니며, ORDER BY보다 먼저 부여되므로 정렬 후 순번이 필요하면 인라인 뷰에서 정렬한 뒤 바깥에서 ROWNUM을 써야 한다.

③ 인덱스와 무관하며 물리적으로 저장되는 컬럼이 아니다. 행의 물리적 위치 정보는 ROWID이다.

④ 사용자가 정의하지 않아도 모든 SELECT에서 바로 사용할 수 있다.

> **보충 개념** WHERE ROWNUM = 2나 WHERE ROWNUM > 1은 첫 행이 ROWNUM 1을 받자마자 탈락하고 다음 행도 다시 1을 받으므로 결과가 나오지 않는다. SQL Server는 TOP, 표준 SQL은 FETCH FIRST N ROWS ONLY로 같은 기능을 구현한다.

37 2과목

다음 중 오류가 발생하는 SQL은?

TBL1

A
100
200
300

TBL2

A	B
100	10
100	20
200	NULL
300	5

① `SELECT A FROM TBL1 WHERE A IN (SELECT A FROM TBL2);`

② `SELECT A, (SELECT SUM(B) FROM TBL2 WHERE TBL1.A = TBL2.A) FROM TBL1;`

③ `SELECT A, (SELECT NVL(B, NULL) FROM TBL2 WHERE TBL1.A = TBL2.A) FROM TBL1;`

④ `SELECT A FROM TBL1 WHERE EXISTS (SELECT 1 FROM TBL2 WHERE TBL1.A = TBL2.A);`

정답 ③

핵심 해설

정답은 ③ 스칼라 서브쿼리가 두 행을 반환하는 SQL이다. SELECT 절의 스칼라 서브쿼리는 행마다 최대 1행 1컬럼만 반환해야 한다.

선택지별 풀이

③ 은 TBL1.A=100일 때 TBL2에서 B=10, B=20 두 행이 반환되므로 '단일 행 서브쿼리가 둘 이상의 행을 반환' 오류가 발생한다. NVL(B, NULL) 자체는 문법상 문제가 없지만 반환 행 수 문제를 해결하지 못한다.

TBL1.A	매칭되는 TBL2.B	반환 행 수
100	10, 20	2 (오류)
200	NULL	1
300	5	1

① IN 서브쿼리는 여러 행을 반환해도 되는 다중 행 비교이므로 정상이며 100, 200, 300이 나온다.

② SUM(B)로 집계해 항상 1행만 반환하므로 정상이다. 결과는 100=30, 200=NULL, 300=5이다.

④ EXISTS는 존재 여부만 판단하므로 매칭 행 수와 무관하게 정상이며 100, 200, 300이 나온다.

> **보충 개념** 스칼라 서브쿼리는 매칭 행이 없으면 NULL을 반환할 뿐 오류가 나지 않지만, 2행 이상이면 오류가 난다. 집계 함수나 ROWNUM = 1 조건 등으로 1행을 보장해야 한다.

---

38 2과목

다음 SQL의 결과로 가장 적절한 것은?

TBL1

COL1
100
200
NULL

TBL2

COL2
100
200
NULL

```
SELECT *
FROM TBL1
WHERE COL1 NOT IN (SELECT COL2
FROM TBL2
WHERE COL2 IN (100, NULL));
```

① 100 ② **200** ③ 100, 200 ④ 결과 없음(공집합)

정답 ②

핵심 해설

정답은 ② 200이다. 안쪽 서브쿼리부터 계산한다. 1단계: WHERE COL2 IN (100, NULL)은 COL2 = 100 OR COL2 = NULL이다. COL2 = NULL은 항상 UNKNOWN이므로 COL2 = 100인 행만 남는다. TBL2의 NULL 행도 비교 결과가 UNKNOWN이라 제외되어 서브쿼리 결과는 100 하나뿐이다. 2단계: 바깥 조건은 COL1 NOT IN (100), 즉 COL1 <> 100이다.

COL1	COL1 <> 100	결과
100	FALSE	제외
200	TRUE	포함
NULL	UNKNOWN	제외

최종 결과는 200 한 행이다.

선택지별 풀이

① 100은 NOT IN (100) 조건에서 FALSE가 되어 제외된다.

③ 100은 서브쿼리 결과에 포함되므로 NOT IN에서 걸러진다.

④ 서브쿼리 결과에 NULL이 들어 있었다면 공집합이 되지만, IN (100, NULL) 조건이 NULL 행을 걸러 서브쿼리 결과에 NULL이 없으므로 공집합이 아니다.

> **보충 개념** IN 목록 안의 NULL은 어떤 행과도 일치하지 않아 사실상 무시되지만, NOT IN의 비교 대상에 NULL이 있으면 결과 전체가 공집합이 된다. 바깥 COL1의 NULL 행은 어떤 비교에서도 TRUE가 될 수 없어 항상 제외된다.

39 2과목

다음 SQL과 결과가 다른 SQL은?

```
SELECT ENAME,
CASE WHEN SAL IS NULL THEN 100 ELSE SAL END AS RESULT
FROM EMP;
```

① `SELECT ENAME, NVL(SAL, 100) AS RESULT FROM EMP;`

② `SELECT ENAME, COALESCE(SAL, 100) AS RESULT FROM EMP;`

③ `SELECT ENAME, DECODE(SAL, NULL, 100, SAL) AS RESULT FROM EMP;`

④ `SELECT ENAME, NVL2(SAL, 100, SAL) AS RESULT FROM EMP;`

정답 ④

핵심 해설

정답은 ④ NVL2(SAL, 100, SAL)이다. 원래 SQL은 SAL이 NULL이면 100, 아니면 SAL을 반환한다. NVL2(expr, a, b)는 expr이 NULL이 아니면 a, NULL이면 b를 반환하므로

선택지별 풀이

④ 는 SAL이 있으면 100, SAL이 NULL이면 NULL을 반환해 정반대가 된다.

SAL	원래 SQL	④ NVL2
3000	3000	100
NULL	100	NULL

① NVL(SAL, 100)은 SAL이 NULL이면 100, 아니면 SAL을 반환하므로 동일하다.

② COALESCE(SAL, 100)은 나열된 값 중 처음으로 NULL이 아닌 값을 반환하므로 동일하다.

③ DECODE는 일반 비교와 달리 NULL과 NULL을 같은 값으로 취급하므로 SAL이 NULL이면 100, 아니면 SAL을 반환해 동일하다.

> **보충 개념** 원래 SQL과 같게 만들려면 NVL2(SAL, SAL, 100)으로 인자 순서를 바꿔야 한다. CASE WHEN SAL = NULL처럼 쓰면 항상 UNKNOWN이 되어 동작하지 않으므로 반드시 IS NULL을 써야 한다.

40 2과목

다음 SQL을 수행한 후 EMP 테이블의 데이터로 가장 적절한 것은?

EMP

EMP_ID	EMP_NAME
10001	KIM

EMP_ID	EMP_NAME
10002	PARK

EMP_TEMP

EMP_ID	EMP_NAME
10001	GIM
10002	LEE

```
MERGE INTO EMP E
USING EMP_TEMP T
ON (E.EMP_ID = T.EMP_ID)
WHEN MATCHED THEN
UPDATE SET E.EMP_NAME = T.EMP_NAME
WHEN NOT MATCHED THEN
INSERT (EMP_ID, EMP_NAME) VALUES (T.EMP_ID, T.EMP_NAME);
```

① (10001, KIM), (10002, PARK)

② (10001, GIM), (10002, LEE)

③ (10001, KIM), (10002, PARK), (10001, GIM), (10002, LEE)

④ (10001, GIM), (10002, PARK)

정답 ②

핵심 해설

정답은 ② (10001, GIM), (10002, LEE)이다. MERGE는 USING 대상(EMP_TEMP)의 각 행을 ON 조건으로 EMP와 비교해 일치하면 UPDATE, 일치하지 않으면 INSERT를 수행한다. EMP_TEMP 행EMP 매칭동작EMP 결과(10001, GIM)10001 있음UPDATE(10001, GIM)(10002, LEE)10002 있음UPDATE(10002, LEE)두 행 모두 MATCHED이므로 INSERT는 일어나지 않고 이름만 갱신된다.

선택지별 풀이

① MATCHED 행은 UPDATE되므로 원래 값 KIM, PARK이 남지 않는다.

③ 두 행 모두 ON 조건에 일치하므로 WHEN NOT MATCHED의 INSERT가 수행되지 않아 행이 4개가 될 수 없다.

④ 10002도 EMP_TEMP와 일치해 LEE로 갱신되므로 PARK이 남지 않는다.

> **보충 개념** ON 조건에 사용한 컬럼(EMP_ID)은 UPDATE SET에서 변경할 수 없다. WHEN MATCHED와 WHEN NOT MATCHED 절은 둘 중 하나만 기술해도 된다.

41 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

```
SELECT REGEXP_SUBSTR('AABBCC', '(A|B)+C'),
REGEXP_SUBSTR('AABBCC', 'AB+C')
FROM DUAL;
```

① AABBCC, ABBC

② AABBC, AABBCC

③ AABBC, ABBC

④ AABBCC, AABBCC

정답 ③

핵심 해설

정답은 ③ AABBC, ABBC이다. REGEXP_SUBSTR는 문자열 왼쪽부터 패턴과 처음 일치하는 부분 문자열을 반환하며, 수량자 +는 가능한 한 길게(탐욕적) 일치시킨다. 첫 번째 '(A|B)+C': 1번째 위치에서 (A|B)+가 A, A, B, B까지 4글자를 소비한 뒤 C 하나와 일치하므로 결과는 AABBC이다. 패턴 끝에 C가 하나뿐이므로 마지막 C는 포함되지 않는다. 두 번째 'AB+C': 1번째 위치는 A 다음 글자가 A라서 B+가 실패한다. 2번째 위치에서 A, B+(BB), C가 일치하므로 결과는 ABBC이다.

선택지별 풀이

① 첫 번째 결과 AABBCC는 C를 두 개 포함하는데, 패턴의 C는 한 번만 일치하므로 틀리다.

② 두 번째 패턴은 A가 하나만 오고 바로 B가 이어져야 하므로 AA로 시작하는 AABBCC와 일치할 수 없다.

④ 두 결과 모두 C가 두 개 포함되어 있고 두 번째는 AA로 시작하므로 틀리다.

> **보충 개념** 정규식은 '가장 왼쪽에서 시작하는 일치'를 먼저 찾고, 그 시작 위치에서 수량자를 가장 길게 적용한다. 시작 위치 탐색과 길이 결정을 구분해서 보면 헷갈리지 않는다.

42 2과목

다음 중 JOIN에 대한 설명으로 가장 적절한 것은 무엇인가?

① CROSS JOIN은 조인 조건을 ON절에 명시해야 한다.

② INNER JOIN은 등가 조인만 가능하다.

③ NATURAL JOIN은 두 테이블에서 이름이 같은 컬럼을 자동으로 찾아 등가 조인을 하는 것이다.

④ USING 절은 두 테이블에서 컬럼명이 서로 다른 경우에도 사용할 수 있다.

정답 ③

핵심 해설

정답은 ③ NATURAL JOIN은 두 테이블에서 이름이 같은 컬럼을 자동으로 찾아 등가 조인을 하는 것이다. NATURAL JOIN은 별도의 조인 조건을 쓰지 않고, 두 테이블에 공통으로 존재하는 동일 이름 컬럼 전체를 대상으로 등가(=) 조인을 수행한다. 이때 공통 컬럼은 결과에 한 번만 나오며, 해당 컬럼에 테이블 별칭(접두사)을 붙일 수 없다.

선택지별 풀이

① CROSS JOIN은 조인 조건 없이 두 테이블의 모든 행을 조합하는 카티션 곱이므로 ON절을 사용하지 않는다.

② INNER JOIN은 ON절에 =뿐 아니라 BETWEEN, >, < 등의 조건을 쓸 수 있어 비등가 조인도 가능하다.

④ USING 절은 두 테이블에 같은 이름으로 존재하는 컬럼만 지정할 수 있으므로, 컬럼명이 서로 다르면 ON절을 사용해야 한다.

> **보충 개념** NATURAL JOIN과 USING 모두 공통 컬럼에 별칭을 붙이면 오류가 난다. 컬럼명이 다르거나 조건을 자유롭게 쓰려면 ON절을 사용한다.

43 2과목

다음 중 SAL(연봉)이 높은 순서로 상위 2명을 조회하는 SQL로 가장 적절한 것은?

① `SELECT EMP_ID FROM EMP ORDER BY SAL DESC;`

② `SELECT EMP_ID, ROW_NUMBER() OVER (ORDER BY SAL DESC) AS RN FROM EMP WHERE RN <= 2;`

③ `SELECT EMP_ID FROM EMP WHERE ROWNUM <= 2 ORDER BY SAL DESC;`

④ `SELECT EMP_ID FROM (SELECT * FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 2;`

정답 ④

핵심 해설

정답은 ④ SELECT EMP_ID FROM (SELECT * FROM EMP ORDER BY SAL DESC) WHERE ROWNUM <= 2;이다. 인라인 뷰 안에서 먼저 SAL 내림차순으로 정렬한 뒤, 바깥 쿼리에서 ROWNUM을 부여하므로 정렬된 결과의 앞 2건, 즉 연봉 상위 2명을 정확히 가져온다.

선택지별 풀이

① 정렬만 하고 행 수를 제한하지 않으므로 전체 사원이 조회된다.

② WHERE절은 SELECT절보다 먼저 처리되므로 SELECT에서 만든 별칭 RN을 WHERE에서 참조할 수 없어 오류가 발생한다. 인라인 뷰로 감싼 뒤 바깥에서 RN <= 2로 걸러야 한다.

③ ROWNUM은 WHERE 처리 시점에 부여되고 ORDER BY는 그 뒤에 실행되므로, 임의의 2건을 먼저 뽑은 뒤 그 2건만 정렬하게 되어 상위 2명이 보장되지 않는다.

> **보충 개념** 처리 순서는 FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY이다. Top-N 조회는 '정렬 후 ROWNUM'이 되도록 인라인 뷰를 사용하거나, 윈도우 함수 결과를 인라인 뷰로 감싸 바깥에서 필터링한다.

44 2과목

다음 테이블에 대해 실행했을 때 오류가 발생하는 SQL은?

```
CREATE TABLE TAB (
COL1 NUMBER        NOT NULL,
COL2 VARCHAR2(10),
COL3 VARCHAR2(10) DEFAULT 'N',
COL4 DATE
);
```

① `INSERT INTO TAB VALUES (1, 'A', 'B');`

② `INSERT INTO TAB (COL1, COL2, COL4) VALUES (1, 'A', SYSDATE);`

③ `INSERT INTO TAB (COL1, COL2) VALUES (1, 'A');`

④ `INSERT INTO TAB VALUES (1, 'A', 'B', SYSDATE);`

정답 ①

핵심 해설

정답은 ① INSERT INTO TAB VALUES (1, 'A', 'B');이다. 컬럼 목록을 생략한 INSERT는 테이블의 모든 컬럼(COL1~COL4, 4개)에 대해 정의 순서대로 값을 제공해야 한다.

선택지별 풀이

① **은 값이 3개뿐이므로 값의 개수가 부족하다는 오류가 발생한다. DEFAULT가 있거나 NULL을 허용하는 컬럼이라도 컬럼 목록을 생략했다면 자동으로 채워지지 않는다.**

② 컬럼 목록 (COL1, COL2, COL4)와 값 3개가 일치한다. 생략된 COL3에는 DEFAULT 'N'이 들어가므로 정상 실행된다.

③ COL1, COL2만 지정했으며 NOT NULL인 COL1에 값이 있으므로 정상이다. COL3은 'N', COL4는 NULL이 된다.

④ 컬럼 목록 없이 4개 컬럼 모두에 순서대로 값을 제공하므로 정상 실행된다.

> **보충 개념** 생략된 컬럼에 DEFAULT 값이 적용되는 것은 컬럼 목록을 명시해 그 컬럼을 빼는 경우뿐이다. 컬럼 목록을 생략하면 값 개수가 컬럼 개수와 반드시 같아야 한다.

45 2과목

다음 두 테이블에 대해 FULL OUTER JOIN을 수행했을 때, 결과 행의 개수는?

TBL1

A
100
200
NULL
NULL

TBL2

A
100
300
NULL

```
SELECT *
FROM TBL1
```

```
FULL OUTER JOIN TBL2
ON TBL1.A = TBL2.A;
```

① 4개 ② 5개 **③ 6개** ④ 7개

정답 ③

핵심 해설

정답은 ③ 6개이다. FULL OUTER JOIN은 조인 조건을 만족하는 행 + 왼쪽에서 매칭되지 않은 행 + 오른쪽에서 매칭되지 않은 행을 모두 반환한다. NULL = NULL은 참이 아니므로 NULL끼리는 조인되지 않는다. 1단계 매칭: TBL1의 100 ↔ TBL2의 100만 일치 → 1행 2단계 TBL1 미매칭: 200, NULL, NULL → 3행 (TBL2 쪽은 NULL) 3단계 TBL2 미매칭: 300, NULL → 2행 (TBL1 쪽은 NULL)

TBL1.A	TBL2.A
100	100
200	NULL
NULL	NULL
NULL	NULL
NULL	300
NULL	NULL

합계 1 + 3 + 2 = 6행이다.

선택지별 풀이

① 4개는 NULL 행을 제외하고 센 경우로, 미매칭 NULL 행도 결과에 포함되므로 틀리다.

② 5개는 NULL끼리 한 번 조인된다고 본 경우 등으로, NULL은 어떤 값과도 같지 않아 성립하지 않는다.

④ 7개는 행 수를 과다 계산한 것으로, 매칭된 100은 한 행으로만 나온다.

> **보충 개념** 조인 조건에서 NULL은 항상 매칭에 실패하므로 OUTER JOIN에서는 각자 미매칭 행으로 따로 나온다.

46 2과목

다음 SQL의 결과로 가장 적절한 것은 무엇인가?

EMP

EMP_ID	DEPT_ID	SAL
100	10	3000
101	10	2500
102	20	NULL
103	20	3200
104	30	2800

```
SELECT (SELECT MAX(SAL) FROM EMP WHERE DEPT_ID = 30)
+ (SELECT MIN(SAL) FROM EMP WHERE DEPT_ID = 20 AND SAL IS NULL)
FROM DUAL;
```

① 2800 **② NULL** ③ 0 ④ 오류 발생

정답 ②

핵심 해설

정답은 ② NULL이다. 두 스칼라 서브쿼리를 각각 계산한 뒤 더한다. 1단계: DEPT_ID = 30인 행은 EMP_ID 104 하나이므로 MAX(SAL) = 2800이다. 2단계: DEPT_ID = 20 AND SAL IS NULL인 행은 EMP_ID 102 하나이고 SAL이 NULL이다. 집계 함수는 NULL을 제외하고 계산하는데 남는 값이 없으므로 MIN(SAL) = NULL이다. 3단계: 2800 + NULL = NULL. 산술 연산에 NULL이 하나라도 포함되면 결과는 NULL이다.

선택지별 풀이

① 2800은 NULL을 0처럼 취급한 결과로, NVL 등으로 변환하지 않는 한 NULL과의 덧셈은 NULL이다.

③ 0은 MIN 결과를 0으로 본 것으로, 집계 대상 값이 없으면 MIN은 0이 아니라 NULL을 반환한다.

④ 스칼라 서브쿼리가 각각 한 행 한 컬럼(집계 결과)을 반환하므로 문법상 오류가 발생하지 않는다.

> **보충 개념** 집계 함수는 대상 행이 없거나 대상 값이 전부 NULL이면 NULL을 반환한다(COUNT 제외, COUNT는 0). 덧셈 결과를 숫자로 받으려면 NVL(…, 0)로 감싸야 한다.

47 2과목

다음 SQL 실행 결과로 출력되는 행의 개수는?

SALES

DEPTNO	PRODUCT	AMT
10	A	100
10	B	200
20	A	300
20	B	400

```
SELECT DEPTNO, PRODUCT, SUM(AMT)
FROM SALES
GROUP BY GROUPING SETS ((DEPTNO, PRODUCT), (DEPTNO), ());
```

① 4개 ② 6개 **③ 7개** ④ 8개

정답 ③

핵심 해설

정답은 ③ 7개이다. GROUPING SETS는 괄호 안의 각 그룹핑 기준별로 GROUP BY한 결과를 UNION ALL한 것과 같다. 1단계 (DEPTNO, PRODUCT): (10,A), (10,B), (20,A), (20,B) → 4행 2단계 (DEPTNO): 10 = 300, 20 = 700 → 2행 3단계 (): 전체 합계 1000 → 1행

DEPTNO	PRODUCT	SUM(AMT)
10	A	100
10	B	200
20	A	300
20	B	400
10	NULL	300
20	NULL	700
NULL	NULL	1000

합계 4 + 2 + 1 = 7행이다.

선택지별 풀이

① 4개는 (DEPTNO, PRODUCT) 그룹만 계산한 경우이다.

② 6개는 빈 괄호 ()가 의미하는 전체 합계 행을 빠뜨린 경우이다.

④ 8개는 PRODUCT별 소계(A, B)까지 포함한 CUBE(DEPTNO, PRODUCT)의 결과 9행과도 다르며, 지정되지 않은 그룹을 더한 계산이다.

> **보충 개념** 이 GROUPING SETS는 ROLLUP(DEPTNO, PRODUCT)와 같은 결과를 낸다. CUBE였다면 PRODUCT별 소계 2행이 추가되어 9행이 된다.

48 2과목

다음 SQL이 실행된 상태에서, 오류가 발생하는 SQL은?

```
CREATE TABLE DEPT (
DEPT_ID NUMBER PRIMARY KEY,
DEPT_NAME VARCHAR2(20)
);
CREATE TABLE EMP (
EMP_ID NUMBER PRIMARY KEY,
EMP_NAME VARCHAR2(20),
DEPT_ID NUMBER,
CONSTRAINT FK_DEPT FOREIGN KEY (DEPT_ID) REFERENCES DEPT(DEPT_ID)
);
INSERT INTO DEPT VALUES (10, '개발팀');
INSERT INTO EMP VALUES (1, 'KIM', 10);
```

① `INSERT INTO EMP VALUES (2, 'LEE', 10);`

② `INSERT INTO EMP VALUES (3, 'PARK', NULL);`

③ `INSERT INTO EMP VALUES (4, 'CHOI', 20);`

④ `DELETE FROM EMP WHERE EMP_ID = 1;`

정답 ③

핵심 해설

정답은 ③ INSERT INTO EMP VALUES (4, 'CHOI', 20);이다. EMP.DEPT_ID는 DEPT.DEPT_ID를 참조하는 외래키(FK_DEPT)이다. 자식 테이블에 값을 넣을 때는 그 값이 부모 테이블에 존재하거나 NULL이어야 한다. DEPT에는 10만 존재하므로 DEPT_ID = 20을 넣으면 부모 키가 없다는 참조 무결성 위반 오류가 발생한다.

선택지별 풀이

① DEPT_ID = 10은 DEPT에 존재하고 EMP_ID 2도 중복되지 않으므로 정상 실행된다.

② 외래키 컬럼에 NOT NULL 제약이 없으므로 NULL은 허용되며 정상 실행된다.

④ 자식 테이블(EMP)의 행을 삭제하는 것은 부모 테이블에 영향을 주지 않으므로 정상 실행된다.

> **보충 개념** 오류가 나는 쪽은 '자식 INSERT/UPDATE 시 부모 값 없음'과 '자식이 참조 중인 부모 DELETE/UPDATE'이다. 부모 삭제 시 동작은 ON DELETE CASCADE, ON DELETE SET NULL 등으로 바꿀 수 있다.

---

49 2과목

다음 SQL의 결과에서 DIFF 컬럼의 값을 SALE_DATE 순서대로 나열한 것으로 가장 적절한 것은?

SALES

SALE_DATE	AMT
2026-01-01	100
2026-01-02	150
2026-01-03	120

```
SELECT SALE_DATE, AMT,
AMT - LAG(AMT) OVER (ORDER BY SALE_DATE) AS DIFF
FROM SALES;
```

① NULL, 50, -30

② 0, 50, -30

③ 100, 50, -30

④ NULL, -50, 30

정답 ①

핵심 해설

정답은 ① NULL, 50, -30이다. LAG(AMT)는 ORDER BY SALE_DATE 기준으로 바로 이전 행의 AMT를 가져오며, 이전 행이 없는 첫 행은 NULL을 반환한다.

SALE_DATE	AMT	LAG(AMT)	DIFF
2026-01-01	100	NULL	NULL
2026-01-02	150	100	50
2026-01-03	120	150	-30

첫 행은 100 - NULL = NULL, 둘째 행은 150 - 100 = 50, 셋째 행은 120 - 150 = -30이다.

선택지별 풀이

② 첫 행을 0으로 본 것으로, LAG의 기본값을 지정하지 않았으므로 NULL이 반환되고 NULL과의 뺄셈도 NULL이다.

③ 첫 행을 100으로 본 것으로, 이전 행을 NULL 대신 0으로 처리했을 때의 결과이며 이 SQL과 다르다.

④ 부호가 반대로, 이전 값 - 현재 값으로 계산한 결과이다. 식은 AMT - LAG(AMT)이므로 현재 값에서 이전 값을 뺀다.

> **보충 개념** LAG(AMT, 1, 0)처럼 세 번째 인자로 기본값을 주면 첫 행의 LAG가 0이 된다. 다음 행 값을 가져오는 함수는 LEAD이다.

50 2과목

다음 SQL을 순서대로 실행했을 때, 테이블에 최종적으로 남아있는 행의 개수는?

```
CREATE TABLE TBL (NUM NUMBER);
INSERT INTO TBL VALUES (1);
SAVEPOINT SP1;
INSERT INTO TBL VALUES (2);
INSERT INTO TBL VALUES (3);
ROLLBACK TO SP1;
INSERT INTO TBL VALUES (4);
COMMIT;
SELECT COUNT(*) FROM TBL;
```

① 0 ② 1 **③ 2** ④ 3

정답 ③

핵심 해설

정답은 ③ 2이다. ROLLBACK TO SAVEPOINT는 해당 저장점 이후의 변경만 취소하고 그 이전 변경은 유지한다. 1단계: INSERT 1 → 테이블 {1} 2단계: SAVEPOINT SP1 지정 3단계: INSERT 2, INSERT 3 → {1, 2, 3} 4단계: ROLLBACK TO SP1 → SP1 이후의 2, 3만 취소 → {1} 5단계: INSERT 4 → {1, 4} 6단계: COMMIT → {1, 4} 확정 따라서 COUNT(*) = 2이다.

선택지별 풀이

① 0은 ROLLBACK TO SP1이 트랜잭션 전체를 취소한다고 본 경우로, 저장점 이전 변경(1)은 유지된다.

② 1은 ROLLBACK 이후에 실행한 INSERT 4를 빠뜨린 경우로, 4는 COMMIT 전에 수행되어 함께 확정된다.

④ 3은 롤백된 2, 3 중 일부가 남았다고 본 경우로, SP1 이후 변경은 모두 취소된다.

> **보충 개념** 저장점으로 롤백해도 트랜잭션은 종료되지 않고 이어진다. 이후 COMMIT 시 저장점 이전 변경과 롤백 뒤 새 변경이 함께 확정된다.